

RTL Design Sherpa

APB RTC Micro-Architecture Specification 1.0

January 4, 2026

Table of Contents

1 Rtc Mas Index..... 9

2 APB RTC - Overview.....10

2.1 Overview.....10

2.1.1 Figure 1.1: RTC Block Diagram.....10

2.2 Functional Description.....11

2.2.1 Time Keeping.....11

2.2.2 Alarm Function.....11

2.2.3 Interrupt Support.....11

2.2.4 Clock Domains.....11

2.2.5 Power Management.....12

2.2.6 Applications.....12

2.2.7 Register Summary.....12

2.3 Timing.....14

2.3.1 Waveform 1.1: Time Register Read.....14

2.3.2 Waveform 1.2: Time Increment with Rollover.....14

2.3.3 Waveform 1.3: Alarm Match.....15

2.3.4 Waveform 1.4: Second-Tick Interrupt.....15

2.4 Navigation.....16

3 APB RTC - Architecture.....17

3.1 Overview.....17

3.1.1 Figure 1.2: RTC Architecture.....17

3.1.2 Module Hierarchy.....17

3.2 Functional Description....19
3.2.1 Time Update Flow....19
3.2.2 Figure 1.3: RTC Time Update....19
3.2.3 Alarm Match Flow....20
3.2.4 Figure 1.4: RTC Alarm Match....20
3.2.5 Time Set Flow....21
3.2.6 Figure 1.5: RTC Time Set....21
3.2.7 Clock Domains....21
4 APB RTC - Register Map....28
4.1 Overview....28
4.2 Register Summary....29
4.3 RTC_CONFIG (0x00)....31
4.3.1 Setting the time (time_set_mode protocol)....32
4.3.2 Reading the time (coherent burst)....35
4.4 RTC_CONTROL (0x04)....36
4.5 RTC_STATUS (0x08)....37
4.6 RTC_ALARM_HOUR (0x2C)....40
4.7 RTC_ALARM_MASK (0x30)....41
4.8 Time Format....42
4.8.1 12-hour mode....42
4.8.2 Out-of-range values....43
4.9 Remaining limitations....44
4.10 History....46
4.11 Navigation....47

5 Retro Legacy Blocks - Product Requirements Document....48
5.1 1. Overview....49
5.1.1 1.1 Purpose....49
5.1.2 1.2 Design Philosophy....49
5.1.3 1.3 Target Applications....49
5.2 2. Implemented Blocks....50
5.2.1 2.1 HPET - High Precision Event Timer....50
5.3 3. Planned Blocks....52
5.3.1 3.1 8259 - Programmable Interrupt Controller (PIC)....52
5.3.2 3.2 8254 - Programmable Interval Timer (PIT)....52
5.3.3 3.3 GPIO - General Purpose I/O....52
5.3.4 3.4 RTC - Real-Time Clock....53
5.3.5 3.5 SMBus Controller....53
5.3.6 3.6 UART - Universal Asynchronous Receiver/Transmitter....53
5.3.7 3.7 SPI Controller....53
5.3.8 3.8 I2C Controller....54
5.3.9 3.9 Watchdog Timer....54
5.3.10 3.10 Power Management / ACPI Controller....54
5.3.11 3.11 IOAPIC - I/O Advanced Programmable Interrupt Controller....54
5.3.12 3.12 Interconnect ID / Version Registers....54
5.4 4. Integration and Wrapper Goals....56
5.4.1 4.1 Individual Block Integration....56
5.4.2 4.2 RLB Wrapper Architecture....56
5.5 5. Design Standards....60

5.5.1 5.1 Reset Handling..... 60
5.5.2 5.2 Register Generation..... 60
5.5.3 5.3 Testbench Architecture..... 60
5.5.4 5.4 FPGA Synthesis Attributes..... 61
5.5.5 5.5 Documentation Requirements..... 61
5.6 6. Quality Metrics..... 62
5.6.1 6.1 Production Readiness Criteria..... 62
5.6.2 6.2 Current Status..... 62
5.7 7. Development Roadmap..... 64
5.7.1 7.1 Phase 1: Foundation (Complete)..... 64
5.7.2 7.2 Phase 2: Core Peripherals (Next 6-9 Months)..... 64
5.7.3 7.3 Phase 3: Advanced Peripherals (9-15 Months)..... 64
5.7.4 7.4 Phase 4: System Integration (15+ Months)..... 64
5.8 8. References..... 65
5.8.1 8.1 External Standards..... 65
5.8.2 8.2 Internal Documentation..... 65
5.8.3 8.3 Block-Specific Documentation..... 65
5.9 9. Success Criteria..... 66
5.9.1 9.1 Individual Block Success..... 66
5.9.2 9.2 Collection Success..... 66
5.9.3 9.3 Long-Term Vision..... 66

List of Figures

Figure 1.1: RTC Block Diagram....10
Figure 1.2: RTC Architecture....17
Figure 1.3: RTC Time Update....19
Figure 1.4: RTC Alarm Match....20
Figure 1.5: RTC Time Set....21

List of Tables

No tables in this document.

List of Waveforms

Waveform 1.1: Time Register Read.....14
Waveform 1.2: Time Increment with Rollover.....14
Waveform 1.3: Alarm Match.....15
Waveform 1.4: Second-Tick Interrupt.....15

1 Rtc Mas Index

Generated: 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

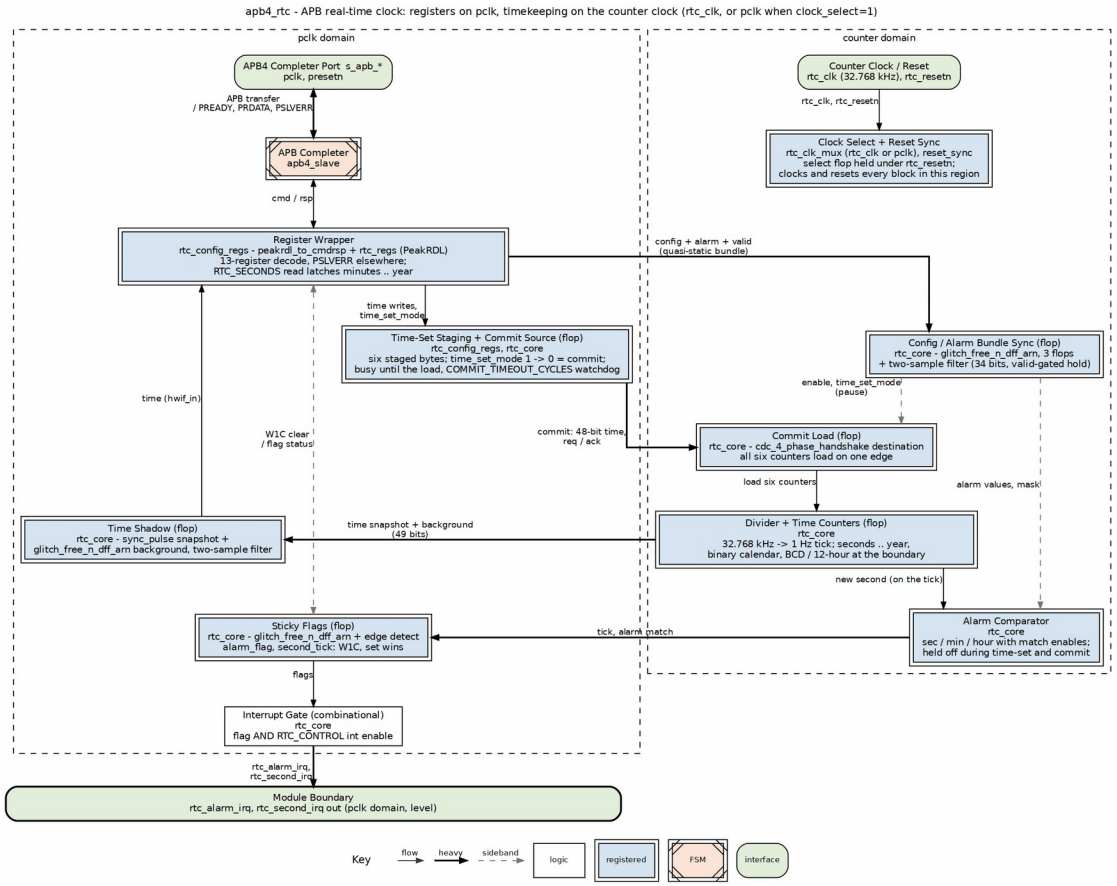
---

2 APB RTC - Overview

2.1 Overview

The APB RTC is a real-time clock controller with an APB slave interface. It keeps time and date, and it raises two kinds of interrupts: a programmable alarm and a fixed 1 Hz tick. The counters live on their own 32.768 kHz clock; the register file lives on pclk; every path between the two is an explicit clock crossing.

2.1.1 Figure 1.1: RTC Block Diagram



RTC Block Diagram

2.2 Functional Description

The feature set is short. Each item below is what the RTL does as of the issue #56 fixes (2026-09-09) — the earlier caveats about broken modes and missing crossings no longer apply.

2.2.1 Time Keeping

- Seconds, minutes, hours in 24-hour or 12-hour (AM/PM) mode — in 12-hour mode bit 7 of RTC_HOURS is the PM flag, in binary and in BCD alike
- Day of month (1-31), month, year (0-99, base year 2000 is hardcoded)
- Leap year calculation (every year divisible by 4 in 2000-2099); the year field wraps 99 to 00 with no century carry
- Binary or BCD register format; the calendar arithmetic runs in binary internally and BCD exists only at the register boundary, so the days-in-month table — leap Februaries included — is right in both formats

2.2.2 Alarm Function

- Single programmable alarm
- Seconds, minutes, hours match with per-field match enables
- Held off while the time is being set or a new time is being loaded
- Asserts on the tick at which the readable time equals the programmed value, never a second later

2.2.3 Interrupt Support

- Alarm match interrupt
- Second-tick interrupt (fixed 1 Hz)
- Both flags are sticky, write-1-to-clear, and a set that lands in the same cycle as a clear wins

2.2.4 Clock Domains

- Counter domain on `rtc_clk`, reset by `rtc_resetn` through a reset synchronizer; APB domain on `pclk`, reset by `presetn` — the two reset independently
- Four crossings between them, each on one of the repo's CDC primitives, so setting the time works at the real 100 MHz : 32.768 kHz ratio and a six-register read is one coherent snapshot; the time-set commit is a four-phase request/acknowledge whose source side is reset by `rtc_resetn` alone (synchronized into pclk), so a `presetn` reset alone can neither lose nor fabricate a transfer and an `rtc_resetn` reset alone resets the link on both

sides; the destination sides of the snapshot and tick synchronizers share that reset, so a `presetn` pulse can neither destroy nor fabricate a snapshot pulse or a flag event either

- A `presetn`-only reset keeps the clock: the counter domain holds its run/enable state and its clock source across it, RTC_CONFIG reads its reset value afterwards, and the counter domain applies the crossed configuration (enable, clock source, mode bits, alarm fields) only once software has written RTC_CONFIG again, so the counters keep counting in the mode they were set in — the time is still correct after a bus reset and `time_valid` is as it was
- `clock_select=1` runs the counters from pclk for test. On a XILINX or SYNTH_PRAGMA target the source mux is a device glitchless cell and the select may be changed live; on every other target, simulation included, it is a plain combinational mux and the select must be changed only with `rtc_enable` low

2.2.5 Power Management

- External 32.768 kHz clock input (no oscillator or power-management logic on chip)

2.2.6 Applications

- System timekeeping
- Scheduled wake-up
- Event timestamping
- Calendar functions
- Alarm clock

2.2.7 Register Summary

Offset	Name	Access	Description
0x00	RTC_CONFIG	RW	Global configuration (enable, hour/BCD/clock mode, time-set)
0x04	RTC_CONTROL	RW	Alarm and interrupt enables
0x08	RTC_STATUS	RO/W1C	Status flags and indicators
0x0C	RTC_SECON	RW	Seconds (0-59)

Offset	Name	Access	Description
	DS		
0x10	RTC_MINUTES	RW	Minutes (0-59)
0x14	RTC_HOURS	RW	Hours (0-23 or 1-12)
0x18	RTC_DAY	RW	Day of month (1-31)
0x1C	RTC_MONTH	RW	Month (1-12)
0x20	RTC_YEAR	RW	Year (0-99, base 2000)
0x24	RTC_ALARM_SEC	RW	Alarm seconds
0x28	RTC_ALARM_MIN	RW	Alarm minutes
0x2C	RTC_ALARM_HOUR	RW	Alarm hours
0x30	RTC_ALARM_MASK	RW	Alarm field match enables

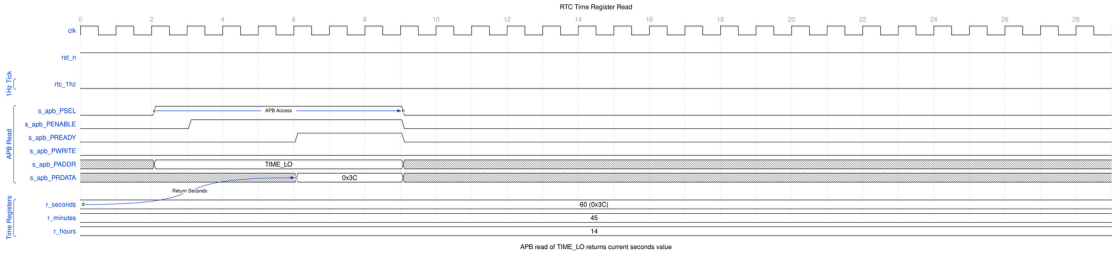
Only these thirteen addresses are decoded; everything else in the 4 KB window is dropped with PSLVERR. See ch05 Register Map for full bit-level definitions, the time-set protocol and the coherent-read contract.

2.3 Timing

The four scenarios below cover normal operation: reading the time, watching it roll over, and the two interrupt sources.

2.3.1 Waveform 1.1: Time Register Read

Reading the time registers returns the current time value.

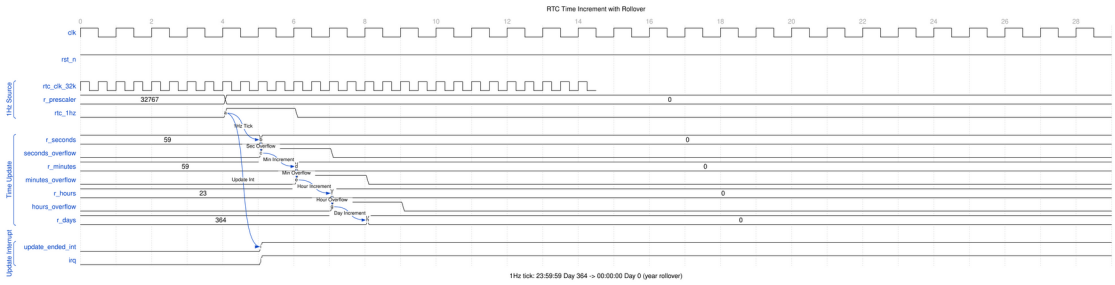


RTC Time Read

Read the six time registers as the burst RTC_SECONDS through RTC_YEAR. The RTC_SECONDS read returns the live seconds and in the same cycle latches minutes, hours, day, month, year, `pm_indicator` and `time_valid`, so the reads that follow belong to the same instant as the seconds value and a tick landing in the middle of the burst cannot mix an old seconds value with a new minutes value. Nothing opens, closes or times out: minutes through year hold what the most recent seconds read captured until the next one, and RTC_SECONDS itself is never latched, so polling it alone returns the live seconds every time — read seconds first. A read reflects a tick about 3-4 pclk cycles after it happens.

2.3.2 Waveform 1.2: Time Increment with Rollover

Shows the cascade of time registers as seconds overflow to minutes, minutes to hours, etc.

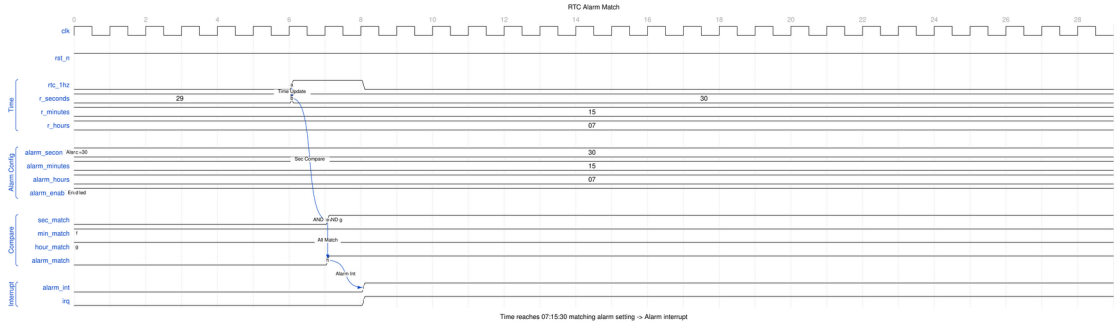


RTC Time Increment

The 1 Hz tick from the 32.768 kHz prescaler triggers the seconds counter. Each overflow cascades to the next register, demonstrating the 23:59:59 to 00:00:00 rollover.

2.3.3 Waveform 1.3: Alarm Match

When the current time matches the alarm setting, an interrupt is generated.

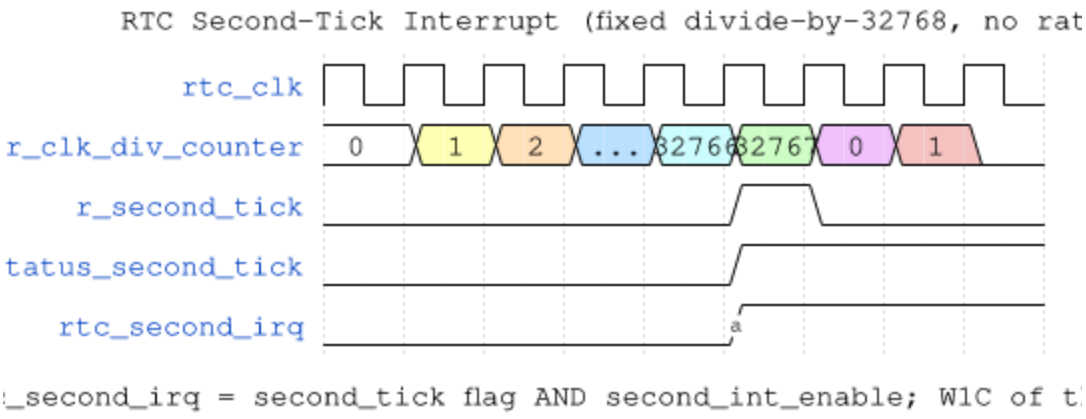


RTC Alarm Match

All configured alarm fields (seconds, minutes, hours) must match simultaneously for the alarm to trigger. The compare is made against the time that becomes readable on that tick, so the alarm asserts at the moment the time registers equal the programmed value, never one second later: an alarm set for hh:mm:ss reads back as hh:mm:ss when the flag sets. In 12-hour mode RTC_ALARM_HOUR is compared as the full byte, PM flag included.

2.3.4 Waveform 1.4: Second-Tick Interrupt

The RTC generates a fixed 1 Hz tick interrupt when enabled by `second_int_enable`.



RTC Periodic Interrupt

The 1 Hz tick is derived from the 32.768 kHz oscillator by a fixed divide-by-32768; there is no programmable rate selector. Each tick sets the `second_tick` status flag and, when enabled, asserts the second-tick interrupt.

2.4 Navigation

Next: 02_architecture.md - Architecture details

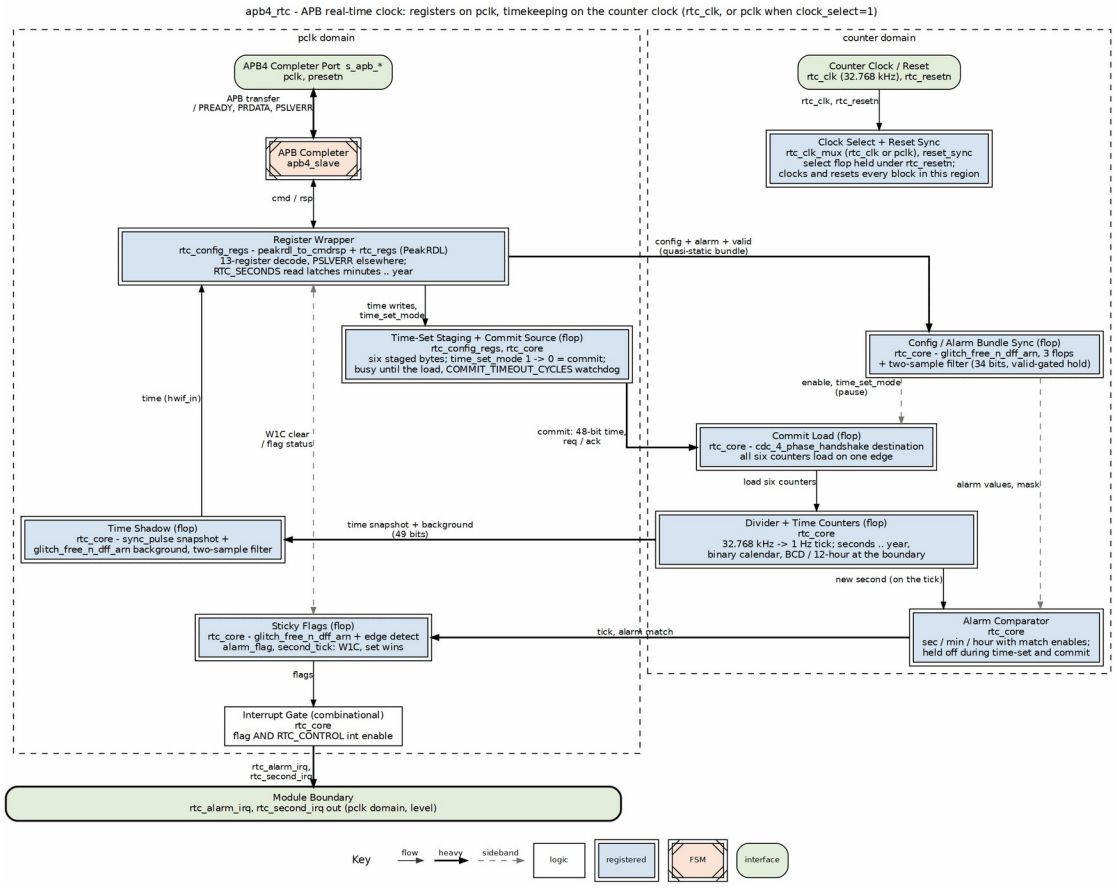
RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

3 APB RTC - Architecture

3.1 Overview

3.1.1 Figure 1.2: RTC Architecture



RTC Architecture

The design splits into two clock domains, drawn side by side: the APB slave, the register wrapper, the time shadow and the sticky flags on pclk at the left, the divider, counters and alarm comparator on the counter clock at the right. The four heavy arrows between the two regions are the only traffic that crosses: the config/alarm bundle and the time-set commit going right, the time snapshot (with its filtered background refresh) and the tick/alarm events coming back.

3.1.2 Module Hierarchy

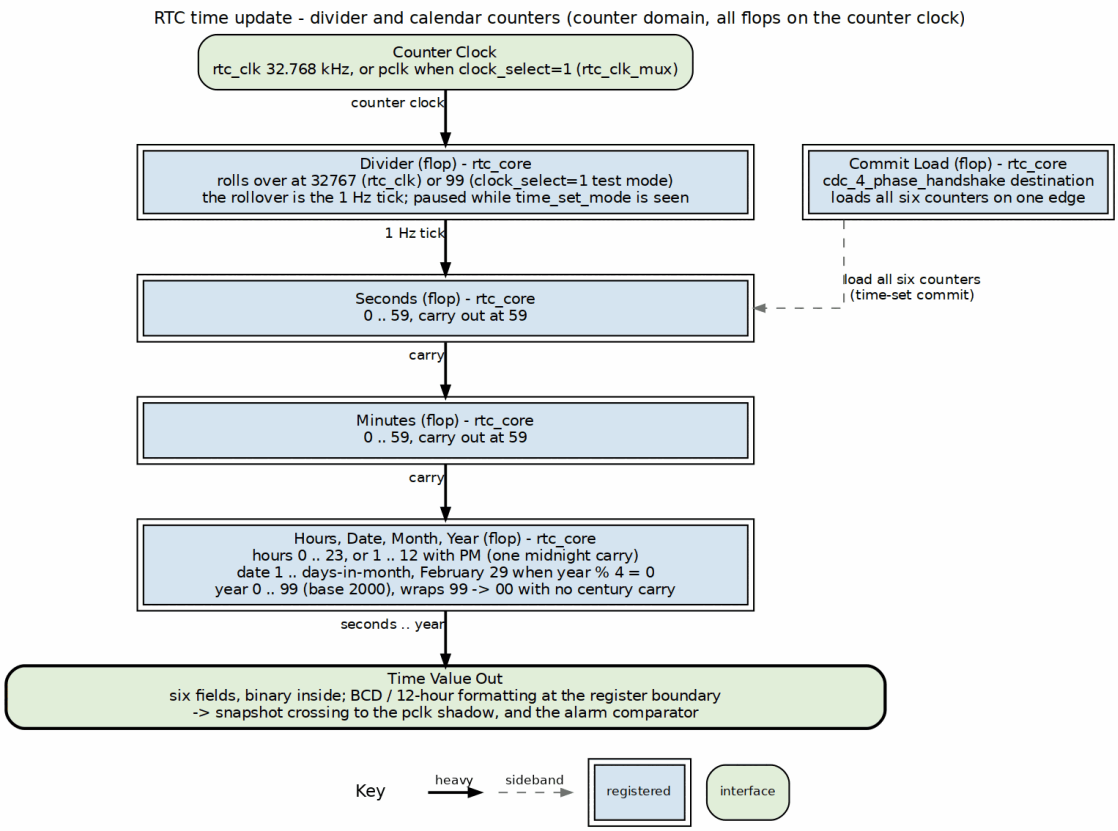
```
apb4_rtc (Top Level)
+-- apb4_slave
+-- rtc_config_regs (Register Wrapper)          [pclk]
|   +-- peakrdl_to_cmdrsp
|   +-- rtc_regs (PeakRDL Generated)
```

```
|   +-- 13-register decode, PSLVERR elsewhere
|   +-- time-set staging and commit
|   +-- seconds-read latch (minutes to year)
|
+-- rtc_core
    +-- rtc_clk_mux (counter clock select: rtc_clk or pclk)
    +-- reset_sync (x2: rtc_resetn onto the counter clock, and into
pclk)
    +-- Divider and Time Counters (seconds to year)   [counter clock]
    +-- Calendar Arithmetic (binary; BCD at the boundary)
    +-- Alarm Comparator                               [counter clock]
    +-- Time Shadow, Sticky Flags, Interrupts          [pclk]
    +-- Clock crossings: glitch_free_n_dff_arn (x3: config/alarm,
        time read, events), cdc_4_phase_handshake (commit),
        sync_pulse (x2: snapshot, commit-load evidence)
```

3.2 Functional Description

3.2.1 Time Update Flow

3.2.2 Figure 1.3: RTC Time Update

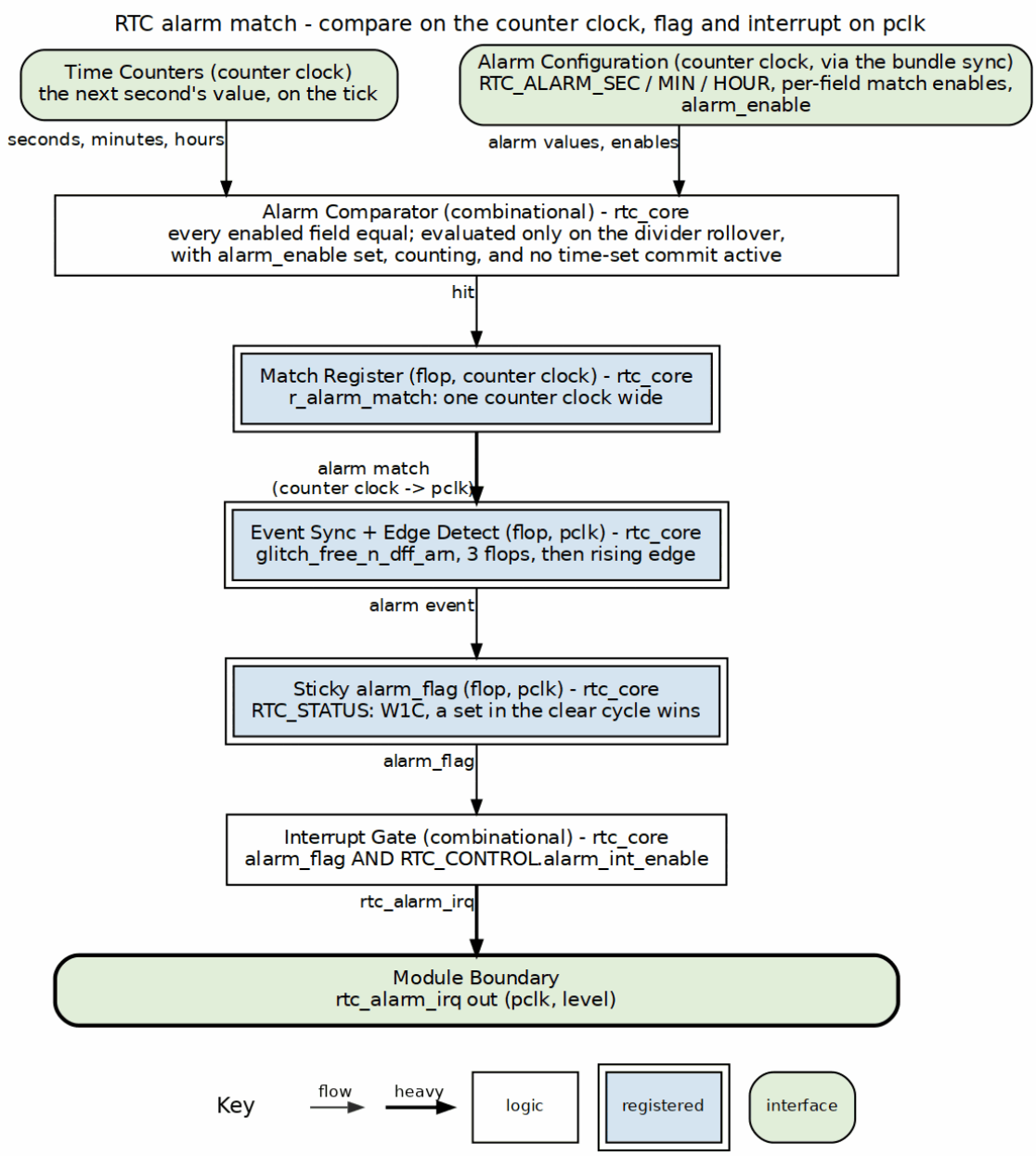


RTC Time Update

The counters advance on the divider rollover edge. The tick is raised on that same edge and marks the new second; the alarm compare is made against that new second — the time that becomes readable on the tick — so the alarm asserts at the moment the readable time equals the programmed value, and when either flag is visible on the APB side the time registers show the second the alarm was set for.

3.2.3 Alarm Match Flow

3.2.4 Figure 1.4: RTC Alarm Match



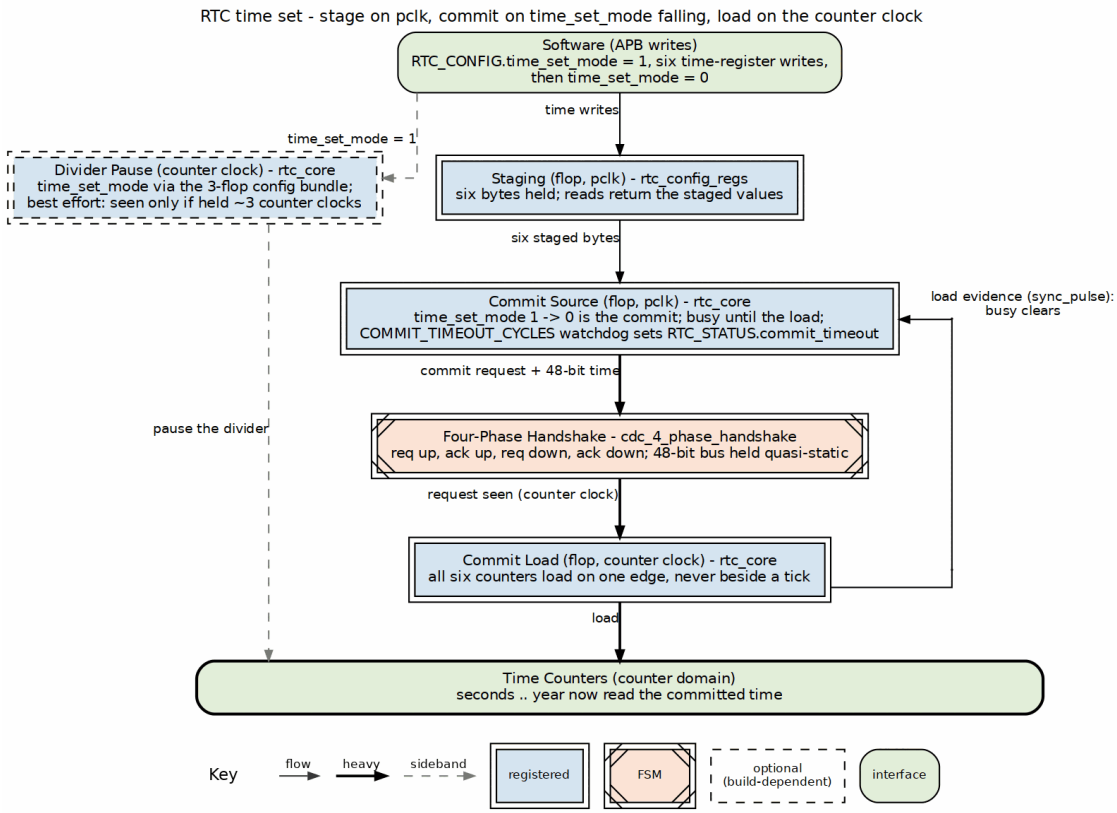
RTC Alarm Match

The comparator is held off while the counter domain sees `time_set_mode` set and for the few counter cycles of a commit, so a counter paused on the alarm value mid-programming does not raise the flag. A commit that lands exactly on the alarm value does not fire the alarm either: alarms fire on a tick, and the next evaluation compares the following second. The alarm values, mask and mode bits reach the comparator through a bit-by-bit crossing, so a change made while

the RTC runs can be sampled as a value that was never programmed for one counter cycle; program the alarm and the mode bits with `alarm_enable` low (or the RTC disabled), then enable.

3.2.5 Time Set Flow

3.2.6 Figure 1.5: RTC Time Set



RTC Time Set

The pause while the time is staged is best effort at the production ratio: `time_set_mode` reaches the counter domain through the 3-flop config crossing, so the counters pause only if the bit is held long enough for the counter clock to sample it — about three counter clocks, ~90 us at 32.768 kHz. A staging window shorter than that is never seen, and the clock keeps running until the commit replaces the time. The commit does not depend on the pause: it brackets the tick, so the counters never take two updates on consecutive counter clocks, and the readable time after the commit is exactly the committed time.

3.2.7 Clock Domains

Domain	Clock	Reset	Holds
APB	`pclk`	`presetn`	register file, address decode, the time shadow

Domain	Clock	Reset	Holds
			software reads, the sticky status flags, the source side of the commit and the destination sides of the snapshot and tick synchronizers (reset by `rtc_resetn` alone, synchronized into pclk, never by `presetn` alone), the config-valid flag that gates the enable/clock-select crossing (set by any RTC_CONFIG write, cleared by `presetn`); the `commit_timeout` flag is the one sticky flag reset by the synchronized `rtc_resetn` instead, with the commit bookkeeping it reports on
Counter	`rtc_clk` (32.768 kHz), or `pclk` when `RTC_CONFIG.clock_select=1`	`rtc_resetn`, release synchronized onto the counter clock by	divider, the six counters, calendar arithmetic, alarm comparator, the

Domain	Clock	Reset	Holds
		reset_sync and additionally held until the clock-select flop has settled, which takes about seven to nine pclk edges after release — so the counter domain cannot leave reset while pclk is stopped; synchronized into pclk it also resets the bus-side commit bookkeeping (pending, busy, the timeout event detector) together with the handshake's source side	run/enable state and the clock-source select (held in a flop under rtc_resetn; the counter domain's reset release waits until that select has been stable for three pclk, so a reset pulse cannot release the domain across a mux switch)

The two resets are independent, in both directions, and the commit handshake's source side is reset by rtc_resetn alone (synchronized into pclk), never by presetn alone — rtc_resetn is synchronized into pclk for that purpose. A presetn reset alone clears the register file and the bus-side state but keeps the clock: the counter domain holds its run/enable state and its clock source across it, so timekeeping continues exactly as before, the time is still correct after the reset and time_valid remains as it was. RTC_CONFIG reads its reset value after the reset, but the counter domain applies a crossed rtc_enable or clock_select only after software has written RTC_CONFIG — a config-valid flag, set by any RTC_CONFIG write and cleared by presetn, crosses inside the 34-bit bundle and the hold loads only a settled word whose valid bit is set, so the valid bit is never seen ahead of the values it qualifies — and the clock-mux select is held in a flop under rtc_resetn, so presetn cannot switch the counter clock; software re-writes RTC_CONFIG to re-establish its configuration, and until it does the counters still format the time with the held BCD/12-hour settings while RTC_CONFIG reads 0, so the time is not to be read before that write. time_set_mode is the one field that is not held: it is a transient control, taken live from the crossing, so a presetn during staging releases the divider pause on its own and drops the staged time. The commit_timeout report is reset with the commit bookkeeping (the synchronized

rtc_resetn), never by presetn, so a bus reset during a stall leaves the report standing while the stall is outstanding and an rtc_resetn that drops the transfer clears it. Any commit already in flight lands intact, never as zeros, and no stale acknowledge can complete a later commit early. Nor does a warm bus reset abort a commit software issued before it: a queued commit still lands with the data captured at its commit pulse, so the time can change shortly after a bus reset because of a request that predates it. An rtc_resetn reset alone returns the counters to the reset default (2000-01-01 00:00:00) and resets the commit link on both sides: a commit that already landed is not replayed, a commit that was still unacknowledged is dropped and software must re-issue it, and a commit issued after the reset works normally. The bus-side commit bookkeeping — pending, busy and the timeout event detector — is reset by either presetn or the synchronized rtc_resetn, the same term that resets the handshake's source side, so a commit staged while rtc_resetn is held low is dropped cleanly: busy does not hang, no timeout is reported for it, the counters read the reset default after release, and software re-issues the commit. This is why the commit crosses on a four-phase request/acknowledge and not a toggle: a toggle stores its state as parity on both sides, and a reset on one side alone would fabricate a transfer the other side never asked for. The completion evidence shares the bookkeeping's reset: the destination sides of the snapshot and tick synchronizers reset only with the synchronized rtc_resetn, so a presetn pulse can neither destroy nor fabricate a snapshot pulse — busy always clears once a commit lands, the register file never publishes a pre-commit time as the answer to a commit, and presetn release cannot manufacture a second_tick or alarm_flag event. Short of a timeout, busy is released only on evidence that the load happened since the commit was accepted — a dedicated load pulse synchronized back into pclk, its destination side on the same reset — together with the shadow showing the committed bytes; a match alone is not enough, so committing the time the clock already shows cannot release busy before the load. The other release is the commit's own window expiring, and then commit_timeout is set to say so. A presetn while busy is set clears the register file, so the six time registers read reset defaults with time_valid=0 until busy clears and the mirror resumes with the committed time.

Every crossing between the domains is explicit, and each uses a primitive from rtl/cdc rather than a hand-rolled synchronizer:

Direction	What crosses	Primitive	Latency
pclk -> counter	config + alarm compare values + the config-valid flag (quasi-static, 34 bits)	glitch_free_n_dff_arn (3 flops) + a two-identical-samples filter on its output; the hold loads only a settled word whose own valid bit is set	5 counter clocks (3 chain + 2 for the filter to see the same word twice); time_set_mode is taken live from the chain output, 3 counter clocks
pclk -> counter	the six staged time bytes + the	cdc_4_phase_handshake	~5 counter clocks (~150 us on a

Direction	What crosses	Primitive	Latency
	commit event	(request/acknowledge, 48-bit quasi-static bus behind it); `RTC_STATUS.commit_timeout` reports an acknowledge missing after `COMMIT_TIMEOUT_CYCLES` pclk (default 65535, about 655 us at 100 MHz), the data held until it lands	32.768 kHz crystal); the report stands while the stall is outstanding — it clears on W1C, or by itself once the commit that finally lands from that slot completes (a retry that replaced the stalled one included), and it is reset with the commit bookkeeping (synchronized `rtc_resetn`), not by `presetn`
counter -> pclk	the coherent time snapshot	`sync_pulse` on the update event, capturing a snapshot register; destination side reset by the synchronized `rtc_resetn`, not `presetn`	3-4 pclk after the counters change
counter -> pclk	commit-load evidence (fires only on a commit load, never on a tick)	a second `sync_pulse`; destination side reset by the synchronized `rtc_resetn`, not `presetn`	3-4 pclk after the load

Direction	What crosses	Primitive	Latency
counter -> pclk	the same time as a background refresh (49 bits: six bytes + `time_valid`)	`glitch_free_n_dff_arn` (3 flops) + a two-identical-samples filter	4-5 pclk after any change
counter -> pclk	`second_tick`, `alarm_match`	`glitch_free_n_dff_arn` + edge detect in pclk; destination side reset by the synchronized `rtc_resetn`, not `presetn`	3-4 pclk

The closed-loop handshake on the write side is what makes the block work at the real clock ratio: a one-pclk strobe sampled by a 32.768 kHz clock is captured about one time in three thousand, which is why the commit is a handshake and not a pulse. It is a four-phase one because the two sides can be reset independently: request rises, the counter side loads and raises acknowledge, request falls, acknowledge falls, and neither side carries parity across a reset. If the counter clock is not running the acknowledge does not come; after `COMMIT_TIMEOUT_CYCLES` pclk (a parameter, default 65535 — about 655 us at 100 MHz) `RTC_STATUS.commit_timeout` sets and the register mirror is released, but the transfer is held — it lands, unchanged, when the counter clock returns. The release is by that transfer's timeout event, never by a persisting timeout condition, so a commit issued meanwhile keeps its staged values visible and the path busy until it lands after the pending one, reports a timeout only if it exceeds the window itself — its own watchdog runs from its commit pulse while it waits to be accepted, so a retry queued behind a link that never returns reports `commit_timeout` again and releases busy rather than hanging — and nothing is delivered twice. A stall is always reported the other way round too: `commit_timeout` sets when the in-flight transfer exceeds the window even if a retry has been queued behind it, and the queued commit keeps busy and its staged values (until its own window expires) and lands after the stalled one when the counter clock returns. Busy never drops while a transfer is pending and inside its own window: a commit accepted in the very cycle the previous timed-out transfer returns to idle keeps busy and its staged values (until its own window expires) until it lands, and no timeout is reported for a transfer that completed. Nor does `commit_timeout` set itself — the bit sets only when a commit issued by software exceeds the window, so a `presetn` pulse, at any time and including during a stall, cannot produce a commit_timeout event after release. The parameter has to exceed the ten or so counter clocks a commit needs at the slowest pclk:rtc_clk ratio, about 30500 pclk at 100 MHz against 32.768 kHz; 0 disables both watchdogs, and a commit on a dead counter clock then holds busy until the clock returns. On the read side the snapshot path is the fast one and the filtered path is the safety net — it needs no event, so a change the pulse never announced still reaches the shadow, and requiring two identical consecutive samples means it can never latch a half-old, half-new word.

For the integrator, the two bundles are multi-bit crossings and the constraints have to say so: the 49-bit time bundle (counter -> pclk) and the alarm/config bundle (pclk -> counter) need their bits to land within one destination clock period of each other, which is `set_max_delay -datapath_only` at the destination period on each bundle — a `set_false_path` alone is not sufficient, since it lets the bits of one word skew across periods and the filter or the comparator can then see a value that was never there. The commit handshake's data bus is quasi-static behind the four-phase and needs only the ordinary multi-cycle treatment against the request bit.

`selected_clk` comes from `rtc_clk_mux`, and the divider target depends on `clock_select` as well. That mux is a device glitchless cell on a XILINX or SYNTH_PRAGMA target – BUFGCTRL with `IGNORE0`/`IGNORE1`, or ALTCLKCTRL – and a plain combinational mux on every other target, simulation included. The select is held in a flop under `rtc_resetn`, so a `presetn` reset cannot switch the counter clock — the counter domain keeps its clock source, and its run/enable state, until software writes RTC_CONFIG again. An `rtc_resetn` pulse does move that flop (clear on assert, reload after release), so the counter domain's reset release is held until the select has been stable for three pclk and the domain never leaves reset across a mux switch. The hold is a one-shot per `rtc_resetn` and a live `clock_select` write never touches the counter reset — it would wipe the time of day, and the mux has already moved by the time a gate could react — which is why the change-only-with-`rtc_enable`-low rule stands on the combinational branch. There, change `RTC_CONFIG.clock_select` only with `rtc_enable` low; switching it while the RTC runs can produce a runt clock pulse, and a runt on the counter clock can also leave an in-flight commit unacknowledged until `commit_timeout` reports it. A device branch lifts that restriction outright. A portable glitch-free mux would not have: break-before-make needs both clocks running, and `clock_select` exists for the case where the crystal may be absent, so a handshake could not switch away from a dead clock at all. In `clock_select=1` mode the crossings still exist, they simply degenerate to a same-clock delay.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

4 APB RTC - Register Map

4.1 Overview

The register block is thirteen 32-bit registers at 0x00-0x30, and exactly those thirteen addresses are decoded. Everything else in the 4 KB window — the reserved slots at 0x34-0x3C as much as anything above 0x3F — is dropped: the write is ignored, the read returns 0, and the access answers with PSLVERR. Nothing aliases. 0x40 is not a second RTC_CONFIG and 0x864 does not reach RTC_ALARM_SEC.

A few behaviors worth knowing before you program this thing. The counters advance on the divider rollover edge; the tick is raised on that same edge and marks the new second, and the alarm compare is made against that new second — the time that becomes readable on the tick — so the alarm asserts at the moment the readable time equals the programmed value, never one second later: an alarm set for hh:mm:ss reads back as hh:mm:ss when `alarm_flag` sets. A read reflects a tick about 3-4 pclk cycles after it happens. With clock_select=1 (test mode) the tick rate is pclk/100, not 1 Hz.

The six time registers are answered from a pclk-domain shadow of the counters, never from the counters themselves, and the shadow only ever takes a complete, coherent time. The register file and the counters are on different clocks; every path between them is one of the four crossings described in ch01, and none of them is a raw sample.

4.2 Register Summary

Offset	Name	Access	Reset	Description
0x00	RTC_CONFIG	RW	0x00000000	Global configuration (enable, hour/BCD/clock mode, time-set)
0x04	RTC_CONTROL	RW	0x00000000	Alarm and interrupt enables
0x08	RTC_STATUS	RO/W1C	0x00000000	Status flags, commit timeout and indicators
0x0C	RTC_SECONDS	RW	0x00	Seconds (0-59)
0x10	RTC_MINUTES	RW	0x00	Minutes (0-59)
0x14	RTC_HOURS	RW	0x00	Hours (0-23, or 1-12 + PM in 12-hour mode)
0x18	RTC_DAY	RW	0x01	Day of month (1-31)
0x1C	RTC_MONTH	RW	0x01	Month (1-12)
0x20	RTC_YEAR	RW	0x00	Year (0-99, base 2000)
0x24	RTC_ALARM_SEC	RW	0x00	Alarm seconds match value
0x28	RTC_ALARM_MIN	RW	0x00	Alarm minutes match value
0x2C	RTC_ALARM_HOUR	RW	0x00	Alarm hours match value
0x30	RTC_ALARM_MASK	RW	0x00	Alarm field match enables

All registers are 32 bits wide. In each register only the low bits listed below are implemented; the remaining bits are reserved (read as 0).

4.3 RTC_CONFIG (0x00)

Bit	Name	Access	Reset	Description
0	rtc_enable	RW	0	Master enable for the RTC (0=disabled, 1=enabled). Reads 0 after a presetn reset, but the counter domain keeps its run state until RTC_CONFIG is written again
1	hour_mode_12	RW	0	Hour format: 0=24-hour, 1=12-hour (AM/PM)
2	bcd_mode	RW	0	Time format: 0=binary, 1=BCD
3	clock_select	RW	0	Time-counter clock: 0=32.768 kHz, 1=system clock (test). Change only with rtc_enable=0. The mux select is held in a flop under rtc_resetn; a presetn reset cannot switch the counter clock
4	time_set_mode	RW	0	1=stage a new time (pauses the divider and the counters once the counter domain samples it), 0=normal operation; the 1-to-0 edge commits
31:5	Reserved	RO	0	Reserved

Note that `hour_mode_12` and the format/clock controls live here, not in RTC_CONTROL. Out of reset the counters run in **binary** mode; BCD is optional and is selected by setting `bcd_mode`. The mode bits, like the alarm values, cross to the counter domain through a three-flop synchronizer and a two-identical-samples filter, and take effect about five counter clocks after the write (three for the chain, two for the filter to see the same word twice). The filter rejects a torn word, so one RTC_CONFIG write lands as the word that was written; but each register write lands on its own, so a sequence of writes is seen one at a time; change `hour_mode_12` and `bcd_mode` with the alarm disabled (or the RTC disabled), then enable.

clock_select drives rtc_clk_mux, and the divider target depends on it too. On a XILINX or SYNTH_PRAGMA target that mux is a device glitchless cell and the select may be changed live; on every other target, simulation included, it is a plain combinational mux and the rule that follows applies. Change it only with rtc_enable low; flipping it while the RTC runs can then produce a runt clock pulse, and a runt on the counter clock can leave an in-flight commit unacknowledged until commit_timeout reports it. Around an rtc_resetn pulse the select flop itself moves the mux twice (it clears on assert and reloads after release), so the counter domain's reset release is held until the select has been stable for three pclk; the counter domain never comes out of reset across a mux switch. That hold is a one-shot per rtc_resetn: a live write of clock_select does not touch the counter reset (it would wipe the time of day, and the mux has already moved by the time a gate could react), which is why the change-only-with- rtc_enable-low rule stands on the combinational branch. A XILINX or SYNTH_PRAGMA target uses a device glitchless cell and is not subject to it.

A presetn reset keeps the clock. The counter domain holds its run/enable state and its clock source across a presetn-only reset: RTC_CONFIG reads its reset value afterwards, but the counter domain applies a crossed configuration — rtc_enable, clock_select, the hour and BCD mode bits, the alarm fields and the alarm enable, everything in the crossed bundle — only after software has written RTC_CONFIG: a config-valid flag, set by any RTC_CONFIG write and cleared by presetn, crosses inside the same 34-bit bundle as the values it qualifies, and the counter domain loads its held copy only from a word that two consecutive samples of the synchronizer agree on and whose own valid bit is set — so the counters keep counting in the mode they were set in rather than switching encoding under a running count, and the valid bit can never be seen ahead of the fields it qualifies. One field is deliberately not held: time_set_mode. It is a transient control, not configuration, and it is taken live from the crossing, so a presetn that lands while a time is being staged releases the divider pause on its own, with no register write required; the staged time is dropped (no commit is generated) and the clock simply keeps running. The mux select is held in a flop under rtc_resetn, so presetn cannot switch the counter clock. Until that write timekeeping continues exactly as before: the time is still correct after a bus reset and time_valid remains as it was. The counters are, however, still formatting the time with the BCD and 12-hour settings from the hold while RTC_CONFIG reads 0 — the register file and the counter domain disagree about the format — so do not read the time until RTC_CONFIG has been re-written. Software re-writes RTC_CONFIG to re-establish its configuration.

4.3.1 Setting the time (time_set_mode protocol)

Outside of time-set mode the six time registers mirror the counter shadow, so a plain write to RTC_SECONDS..RTC_YEAR is overwritten by the live time on the next update. To set the time:

1. Write RTC_CONFIG with time_set_mode = 1, and the desired hour_mode_12 and bcd_mode. The six time registers stop mirroring the counter: they are staging registers now. Once the counter domain samples the bit — about three counter clocks, ~90 us at 32.768 kHz — the divider and the counters pause: no tick, no second interrupt, no alarm evaluation, no counter advance. rtc_enable does not need to be set for the commit to land.

2. Write RTC_SECONDS, RTC_MINUTES, RTC_HOURS, RTC_DAY, RTC_MONTH and RTC_YEAR, in any order, at any speed. Each write lands in its own register and nothing crosses to the counter domain yet, so back-to-back writes cannot lose a field and an individual write never loads a partial time. Reading a time register in this window returns the staged value.
3. Write RTC_CONFIG with `time_set_mode = 0`. That falling edge is the commit: all six bytes cross to the counter domain as one four-phase request/acknowledge transfer and load the counters on a single counter-clock edge. The load restarts the divider, so the first tick after a set comes exactly one divider period after the load — 32768 counter clocks in production, 100 in `clock_select=1` test mode — the same as every steady-state second.

The commit takes about five counter clocks to land — roughly 150 us on a 32.768 kHz crystal — and until the loaded time is visible in the pclk shadow the six registers keep returning the staged values, so a readback in that interval shows the time you asked for rather than the pre-commit time; `RTC_STATUS.pm_indicator` follows bit 7 of the staged hours for as long as the staged values are the ones being read. Busy low means one of two things, and `commit_timeout` tells them apart: with `commit_timeout` clear, the committed time has landed in the counters and is visible in the shadow (the release requires evidence that the load happened since this commit was accepted, a load pulse synchronized back into pclk, not merely that the staged bytes match what the shadow already showed); with `commit_timeout` set, the commit's window expired and busy was released while the commit is still held — committing the time the clock already shows cannot release busy before the load, so a reader polling busy never sees the time step backwards. A `presetn` pulse while busy is set clears the register file, so the six time registers read their reset defaults — not the staged values — until busy clears and the mirror resumes with the committed time; `RTC_STATUS.time_valid` reads 0 for that window and is the signpost. There is no separate commit register; the falling edge was chosen so the published map did not change.

If the counter clock is not running the acknowledge does not arrive. A commit that is not acknowledged within `COMMIT_TIMEOUT_CYCLES` pclk (a parameter, default 65535, about 655 us at 100 MHz) sets `RTC_STATUS.commit_timeout` (sticky, W1C) and releases the register mirror — the six time registers go back to showing the shadow — but the transfer is held: its data stays on the request side and it lands, unchanged, whenever the counter clock returns. `commit_timeout` therefore means "not acknowledged in time; read the time back to see whether it landed", not "the time was not set". The report stands while the stall is outstanding: it clears on W1C, or by itself once the stalled commit finally completes, so a bit that reads 1 always means a commit is still unacknowledged. The flag follows the transfer it reports: it is reset by the synchronized `rtc_resetn` together with the rest of the commit bookkeeping, not by `presetn`, so a bus reset during a stall leaves the report standing while the stall is outstanding, and an `rtc_resetn` that drops the stalled transfer clears the report with it, without a W1C. The release is by the timeout event of the transfer that timed out, never by a persisting timeout condition: a commit issued while a timed-out transfer is still pending keeps its staged values visible and the commit path busy until it lands, after the pending one, and does not itself report a timeout unless it also

exceeds the window. That window is the queued commit's own: a watchdog counted from its commit pulse while it waits to be accepted, so a retry queued behind a link that never returns (a dead crystal) reports `commit_timeout` again after `COMMIT_TIMEOUT_CYCLES` and releases busy and the mirror, while staying queued to land when the clock returns — busy cannot hang and a stall cannot go unreported however many times software retries. Nothing is delivered twice. A stall is always reported, though: `commit_timeout` sets when the in-flight transfer exceeds the window even if a retry has been queued behind it, and the queued commit keeps busy and its staged values and lands after the stalled one when the counter clock returns. Busy never drops while a transfer is pending and inside its own window: a commit accepted in the very cycle the previous timed-out transfer returns to idle keeps busy and its staged values until it lands or its own window expires, and no timeout is reported for a transfer that completed. `commit_timeout` never sets itself: the bit sets only when a commit issued by software exceeds the window, so a `presetn` pulse — at any time, including during a stall — cannot produce a commit_timeout event after release. For the integrator, `COMMIT_TIMEOUT_CYCLES` must exceed the ten or so counter clocks a commit needs at the slowest pclk:rtc_clk ratio, about 30500 pclk at 100 MHz against 32.768 kHz; the default is roughly twice that. A value of 0 disables both watchdogs (the in-flight one and the queued commit's), and then a commit on a dead counter clock holds busy and the staged values until the clock returns.

The handshake is four-phase rather than a toggle because the two domains reset independently: a toggle keeps its state as parity on both sides, and a one-sided reset would fabricate a transfer. The handshake's source side is reset by `rtc_resetn` alone (synchronized into pclk), never by `presetn` alone — `rtc_resetn` is synchronized into pclk for that purpose. So a `presetn` reset alone clears the register file and the bus-side state but keeps the clock: the counter domain holds its run/enable state and its clock source, timekeeping continues exactly as before, and the crossed enable and clock select are applied again only once software has written RTC_CONFIG. Any commit already in flight lands intact, never as zeros, and no stale acknowledge can complete a later commit early. Nor does a warm bus reset abort a commit software issued before it: a queued commit still lands with the data captured at its commit pulse, so the time can change shortly after a bus reset because of a request that predates it. The completion evidence shares the bookkeeping's reset — the destination sides of the snapshot and tick synchronizers reset only with the synchronized `rtc_resetn` — so a `presetn` pulse can neither destroy nor fabricate a snapshot pulse: busy always clears once a commit lands, the register file never publishes a pre-commit time as the answer to a commit, and `presetn` release cannot manufacture a `second_tick` or `alarm_flag` event. An `rtc_resetn` reset alone returns the counters to the reset default and resets the commit link on both sides, so a commit that was still unacknowledged when it asserted is dropped and software must re-issue it; a commit issued after the reset works normally, and a commit that already landed is not replayed. The bus-side commit bookkeeping — pending, busy and the timeout event detector — is reset by either `presetn` or the synchronized `rtc_resetn`, the same term that resets the handshake's source side, so a commit staged while `rtc_resetn` is held low is dropped cleanly: busy does not hang, no timeout is reported for it, the counters read the reset default after release, and software re-issues the commit.

The pause while the time is staged is best effort at the production ratio. `time_set_mode` reaches the counter domain through the 3-flop config crossing, so the divider and the counters pause — no `second_tick`, no second interrupt, no alarm evaluation, no counter advance — only if the bit is

held long enough for the counter clock to sample it, about three counter clocks (~90 us at 32.768 kHz). A sub-microsecond staging window is not seen and the clock keeps running until the commit replaces the time. The commit does not depend on the pause: it brackets the tick, so the counters never take two updates on consecutive counter clocks, the readable time after the commit is exactly the committed time, and the first tick after it comes exactly one divider period after the load, the same as every steady-state second.

4.3.2 Reading the time (coherent burst)

Read the time as the burst RTC_SECONDS, RTC_MINUTES, RTC_HOURS, RTC_DAY, RTC_MONTH, RTC_YEAR — seconds first. A read of RTC_SECONDS returns the live shadow seconds and, in that same cycle, latches minutes, hours, day, month, year, `pm_indicator` and `time_valid`, so the reads that follow belong to the same instant as the seconds value and a tick landing in the middle of the burst cannot produce a mixed answer. There is nothing to open or close and nothing times out: the five latched registers simply hold what the most recent seconds read captured, for as long as it takes, until the next seconds read replaces it.

RTC_SECONDS itself is never latched, so polling it alone in a tight loop returns the live seconds every time. Minutes through year always return the values latched by the most recent seconds read — reading one of them without a preceding RTC_SECONDS read gives whatever the last seconds read captured, however long ago, which is why software reads seconds first. `RTC_STATUS.pm_indicator` and `time_valid` are latched with the five, so the hours and the PM bit read inside one burst belong to the same second. The time you read is the time as of the last second boundary, 3-4 pclk after that boundary.

4.4 RTC_CONTROL (0x04)

Bit	Name	Access	Reset	Description
0	alarm_enable	RW	0	Enable alarm comparison
1	alarm_int_enable	RW	0	Enable interrupt on alarm match
2	second_int_enable	RW	0	Enable interrupt every second (1 Hz tick)
31:3	Reserved	RO	0	Reserved

The interrupt outputs are levels: `rtc_alarm_irq` is `alarm_flag AND alarm_int_enable`, `rtc_second_irq` is `second_tick AND second_int_enable`. Clearing the flag drops the interrupt.

4.5 RTC_STATUS (0x08)

Bit	Name	Access	Reset	Description
0	alarm_flag	W1C	0	Alarm triggered; write 1 to clear
1	second_tick	W1C	0	1 Hz tick occurred; write 1 to clear
2	time_valid	RO	0	Set once the counters have been loaded by a time-set commit or have ticked at least once — a never-programmed RTC reports valid one second after enable, counting from the reset default 2000-01-01. Latched by the RTC_SECONDS read along with minutes through year. Not disturbed by a presetn reset: the counter domain keeps running, so the time and this bit are as they were
3	pm_indicator	RO	0	12-hour mode: 0=AM, 1=PM. Mirrors RTC_HOURS bit 7; valid in both binary and BCD counting, reads 0 in 24-hour mode. Latched by the RTC_SECONDS read along with minutes through year; while time_set_mode is set it reflects bit 7 of the staged hours
4	commit_ti	W1C	0	A time-set commit was

Bit	Name	Access	Reset	Description
	meout			not acknowledged by the counter domain within COMMIT_TIMEOUT_CYCLES pclk (default 65535; counter clock not running). The transfer is held and lands, unchanged, when the counter clock returns — read the time back to see whether it has. Sets only when a commit issued by software exceeds the window; a presetn pulse cannot produce it after release. Write 1 to clear. The bit reports an outstanding stall: it is cleared by W1C, and it retires itself when the commit that finally lands from that slot completes (a retry that replaced the stalled one included). Reset by the synchronized rtc_resetn with the rest of the commit bookkeeping, not by presetn: a bus reset during a stall leaves the report standing, and an rtc_resetn that drops the stalled transfer clears it
others	Reserved	RO	0	Reserved

`alarm_flag` and `second_tick` are sticky flags held in the pclk domain. Each is set by a single-cycle event: the counter-domain tick and match levels are one counter clock wide (about 30 us on

a real crystal), and they are synchronized and edge-detected in pclk before they touch the flag, so a wide source level cannot re-arm a flag one cycle after software cleared it. A W1C that lands in the same cycle as a new event leaves the flag set — set wins over clear, so an interrupt is never silently dropped — and the register mirror follows the flag in the same cycle as the clear, so a read after the W1C sees the flag low. The synchronizers' destination sides reset with the synchronized `rtc_resetn`, not `presetn`, so a `presetn` release cannot manufacture a `second_tick` or `alarm_flag` event — a flag seen after a bus reset is a tick or a match that actually happened.

4.6 RTC_ALARM_HOUR (0x2C)

RTC_ALARM_HOUR is compared as the full byte against RTC_HOURS. In 12-hour mode that means bit 7 is the PM flag and must be programmed along with the 1-12 hour in bits 6:0: 3 PM is 0x83 in binary and 0x93 in BCD, while 0x03 and 0x13 are 3 AM. In 24-hour mode bit 7 is 0 on both sides.

The alarm values and mask cross to the counter domain as one quasi-static bundle behind a two-identical-samples filter, and are consumed on a tick. The filter rejects a torn word, but the three alarm registers are separate writes and land one at a time, so an alarm reprogrammed while the RTC runs can match on a partly updated value; program the alarm (and the mode bits) with `alarm_enable` low, or with the RTC disabled, and then enable.

4.7 RTC_ALARM_MASK (0x30)

Bit	Name	Access	Reset	Description
0	sec_match_en	RW	0	1=compare seconds for alarm, 0=don’t care
1	min_match_en	RW	0	1=compare minutes for alarm, 0=don’t care
2	hour_match_en	RW	0	1=compare hours for alarm, 0=don’t care
31:3	Reserved	RO	0	Reserved

The mask bits are active-high *match enables*, not “ignore” bits. Out of reset all three are 0, so every field is a don’t-care and an enabled alarm matches every second. Set the relevant `*_match_en` bits for the fields that must match.

The alarm is not evaluated while the counter domain sees `time_set_mode` set — the divider is paused, so there is no tick to evaluate it on — nor during the few counter clocks of a commit, so a counter paused on the alarm value mid-programming does not raise the flag. A commit that lands exactly on the alarm value does not fire the alarm either: alarms fire on a tick, and the next evaluation compares the following second.

4.8 Time Format

Time/date values are stored in binary by default. When `bcd_mode` (RTC_CONFIG bit 2) is set, the same fields are presented in BCD:

Field	Binary	BCD
Seconds	0-59	0x00-0x59
Minutes	0-59	0x00-0x59
Hours (24-hour)	0-23	0x00-0x23
Hours (12-hour)	1-12, bit 7 = PM	0x01-0x12, bit 7 = PM
Day of month	1-31	0x01-0x31
Month	1-12	0x01-0x12
Year	0-99	0x00-0x99

All calendar arithmetic is done in binary internally; BCD exists only at the register boundary. The days-in-month table is therefore the same in both formats: 31 for January, March, May, July, August, October and December, 30 for April, June, September and November, and 28 or 29 for February. The year is two digits with base 2000, and the leap rule is every year divisible by 4 — exact for 2000-2099, since 2000 is a leap year and the century exception does not arise before 2100. The year wraps 99 to 00 with no century carry.

4.8.1 12-hour mode

In 12-hour mode the hour occupies bits 6:0 (1-12, or 0x01-0x12 in BCD) and bit 7 is the PM flag, in both binary and BCD counting. The sequence is:

Hours	PM	Next hours	Next PM	Day carry
11	0	12	1	no (11:59:59 AM -> 12:00:00 PM)
11	1	12	0	yes (11:59:59 PM -> 12:00:00 AM, midnight)
12	x	1	x	no (12:59:59 ->

Hours	PM	Next hours	Next PM	Day carry
				1:00:00, no toggle)

AM/PM toggles on the 11 -> 12 transition, 12 -> 1 does not toggle, and the day carries once per day at 11:59:59 PM -> 12:00:00 AM. `RTC_STATUS.pm_indicator` mirrors bit 7 and is valid in both formats.

4.8.2 Out-of-range values

Loading an out-of-range value (day 40, hours 90) is software's responsibility and software should not do it. The counters accept it and the arithmetic stays defined — every limit comparison is a greater-or-equal, never an equality — but the calendar is only meaningful again once the field wraps. In BCD mode an out-of-range load is clamped to the field's maximum on the next tick rather than silently rewritten to a wrong digit.

4.9 Remaining limitations

None of these is a defect in the block; they are the edges of what it does.

- `clock_select` drives a combinational clock mux on any target without a device glitchless cell – that is, everywhere except XILINX and SYNTH_PRAGMA, simulation included – so there it must be changed with `rtc_enable` low.
- The alarm registers land in the counter domain one write at a time; program the alarm (or the mode bits) with the alarm (or the RTC) disabled, then enable.
- Time-set mode pauses the divider only once the counter domain samples it, about three counter clocks; a shorter staging window is not seen and the clock keeps running until the commit replaces the time. The commit brackets the tick either way, so the counters never take two updates on consecutive counter clocks.
- A commit the counter domain does not acknowledge within `COMMIT_TIMEOUT_CYCLES` is reported in `RTC_STATUS.commit_timeout`, and its data is held and lands unchanged when the counter clock returns; the flag means “not acknowledged in time”, not “not set”, and it releases the staged values by that transfer’s timeout event, not by a persisting condition. A stall is always reported: the flag sets when the in-flight transfer exceeds the window even if a retry has been queued behind it, and the queued commit keeps busy and its staged values (until its own window expires) and lands after the stalled one.
- A commit that lands exactly on the alarm value does not fire the alarm; alarms fire on a tick, and the next evaluation compares the following second.
- The resets are independent: `presetn` alone keeps the clock — the counter domain holds its run/enable state and its clock source, the time and `time_valid` are still correct afterwards, and a crossed `rtc_enable` or `clock_select` is applied again only once software re-writes RTC_CONFIG (`time_set_mode` is the exception: it is not held, so a bus reset during staging releases the pause by itself and drops the staged time); the `commit_timeout` report survives a bus reset while its stall is outstanding; a commit in flight lands intact, and a commit queued before the reset still lands with the data captured at its commit pulse, so the time can change shortly after a bus reset because of a request that predates it. `rtc_resetn` alone resets the

commit link on both sides and the bus-side commit bookkeeping, so a commit staged while it is held low is dropped without hanging busy or reporting a timeout, and must be re-issued.

- Out-of-range loads are accepted, not rejected; in BCD mode they are clamped to the field maximum on the next tick.
- Clock calibration, battery-backup domain handling, day-of-week and century rollover past 2099 are not implemented. In particular the counter domain's reset release waits for the clock-select settle one-shot, a pclk flop, so an `rtc_resetn` pulse while `pclk` is stopped leaves the counters in reset until pclk returns.

Both of RLB TASK-010 (was RLB-010)'s deferred items have since landed: the shared CDC primitives are verilator `-Wall` clean as of dc4ea9db7, and the counter-clock source mux is now `rtc_clk_mux`, which supplies a device glitchless cell on XILINX and SYNTH_PRAGMA and the original combinational mux everywhere else. RLB TASK-010 (was RLB-010) is tracked in `vault/tracking/projects/components/legacy-ip/retro_legacy_blocks/`.

4.10 History

The clock crossings, the atomic time-set commit, the seconds-latched read, the sticky single-event status flags, the binary-internal calendar, the 12-hour sequencing, the strict decode and the `rtc_resetn` path were all fixed on 2026-09-09 under GitHub issue #56.

4.11 Navigation

Back to: RTC Specification Index

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

5 Retro Legacy Blocks - Product Requirements Document

Component: Retro Legacy Blocks (RLB) - Production-Quality Legacy Peripherals **Version:** 1.0
Status: Active Development - HPET Production Ready **Last Updated:** 2025-10-29

5.1 1. Overview

5.1.1 1.1 Purpose

The Retro Legacy Blocks (RLB) component provides production-quality implementations of legacy peripheral blocks based on proven peripheral designs. These blocks are designed to be reusable, well-tested, and suitable for both FPGA and ASIC implementation.

5.1.2 1.2 Design Philosophy

“Retro” - Proven Architectures: - Implements time-tested peripheral designs from successful platforms - Focuses on simplicity, reliability, and well-understood behavior - Prioritizes production-readiness over experimental features

“Legacy” - Time-Tested Interfaces: - Based on proven peripheral interface specifications - Suitable for systems requiring retro-compatible peripheral compatibility - APB-based interface for easy integration

“Blocks” - Modular Collection: - Each peripheral is independent and self-contained - Clear separation between different blocks (rtl/hpet/, rtl/gpio/, etc.) - Can be used individually or wrapped into integrated subsystem

5.1.3 1.3 Target Applications

- Retro-compatible platform compatibility layers
- Embedded systems requiring legacy peripheral interfaces
- FPGA-based system emulation
- Educational platforms demonstrating classic peripheral designs
- Mixed-vintage SoC integration (modern + legacy interfaces)

5.2 2. Implemented Blocks

5.2.1 2.1 HPET - High Precision Event Timer

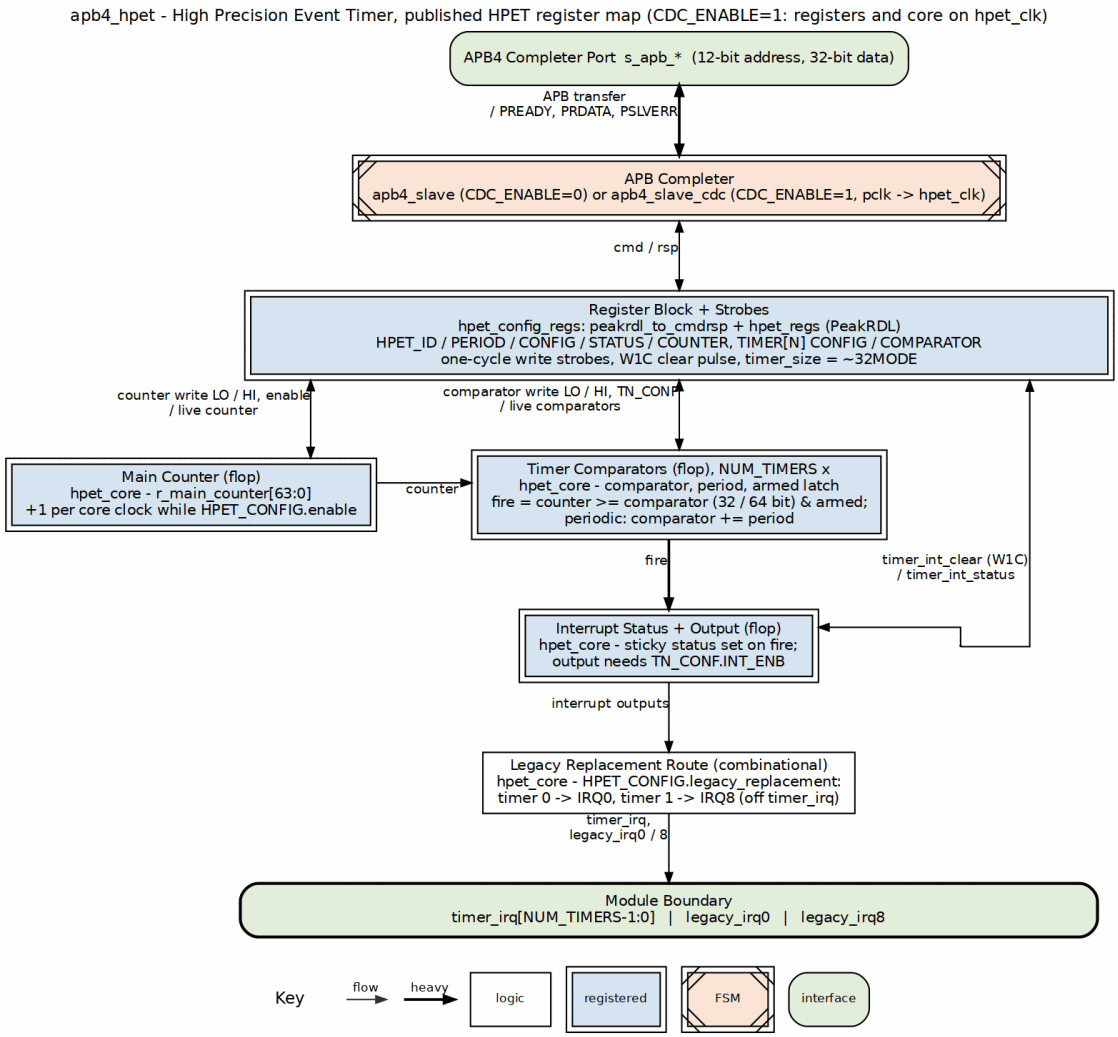
Status: Production Ready (5/6 configurations 100% passing) **RTL Location:** `rtl/hpet/` **Documentation:** `docs/hpet_mas/`

Key Features: - Configurable timer count: 2, 3, or 8 independent timers - 64-bit main counter for high-resolution timestamps - 64-bit comparators per timer - Operating modes: One-shot and periodic - Clock domain crossing: Optional CDC for timer/APB clock independence - APB4 interface: Standard AMBA APB protocol - PeakRDL integration: Register map generated from SystemRDL specification

Applications: - System tick generation - Real-time OS scheduling - Precise event timing - Performance profiling - Watchdog timers - Multi-rate timing domains

Test Coverage: - 6 configurations tested (2/3/8 timers, CDC on/off) - 5/6 configurations at 100% pass rate - 1 configuration at 92% (minor stress test timeout) - 12 test cases per configuration (gate/func/full)

Architecture:



Design Highlights: - Reset macro standardization (FPGA-friendly) - Per-timer data buses prevent corruption - Edge-triggered register write strobes (not level) - W1C status register for interrupt clearing - Optional asynchronous clock domains with handshake CDC

See: `docs/hpet_mas/hpet_mas_index.md` for complete HPET specification

5.3 3. Planned Blocks

5.3.1 3.1 8259 - Programmable Interrupt Controller (PIC)

Status: Implemented (see dv/tests and the block MAS; per-block defects tracked in known_issues/) **Priority:** High **Address:** `0xFEC0_1000 - 0xFEC0_1FFF` (4KB window, master of the cascaded pair)

Planned Features: - 8259A-compatible register interface - 8 interrupt request (IRQ) inputs - Cascadable (master/slave configuration) - Priority resolver (fixed and rotating priority) - Edge and level triggered modes - Interrupt mask register - End-of-Interrupt (EOI) handling - APB register interface

Applications: - Legacy interrupt management - PC-compatible systems - Hardware interrupt aggregation - Priority-based interrupt handling - Cascaded multi-level interrupt systems

5.3.2 3.2 8254 - Programmable Interval Timer (PIT)

Status: Implemented (see dv/tests and the block MAS; per-block defects tracked in known_issues/) **Priority:** High **Address:** `0xFEC0_2000 - 0xFEC0_2FFF` (4KB window)

Planned Features: - 8254-compatible register interface - 3 independent 16-bit counters - 6 programmable counter modes - Binary and BCD counting - Read-back command - Configurable clock input - Interrupt/output generation per counter - APB register interface

Counter Modes: - Mode 0: Interrupt on terminal count - Mode 1: Hardware retriggerable one-shot - Mode 2: Rate generator - Mode 3: Square wave mode - Mode 4: Software triggered strobe - Mode 5: Hardware triggered strobe

Applications: - System tick generation - Periodic timer interrupts - Square wave generation - Event counting - Legacy PC timer compatibility

5.3.3 3.3 GPIO - General Purpose I/O

Status: Implemented (see dv/tests and the block MAS; per-block defects tracked in known_issues/) **Priority:** Medium **Address:** `0xFEC0_7000 - 0xFEC0_7FFF` (4KB window)

Planned Features: - Configurable pin count (8, 16, 32 pins) - Per-pin direction control (input/output/bidirectional) - Input debouncing logic - Interrupt generation (rising/falling/both edges, level) - Output drive strength configuration - Pull-up/pull-down control - APB register interface

Applications: - LED control - Button inputs - Hardware control signals - Chip-select generation - Status monitoring

5.3.4 3.4 RTC - Real-Time Clock

Status: Implemented (apb4_rtc + MAS; see docs/rtc_mas – binary 24-hour timekeeping validated; BCD/12-hour modes and the production-mode time-set path are broken, tracked in issue #56) **Priority:** Medium **Effort:** 3-4 weeks **Address:** `0xFEC0_3000 - 0xFEC0_3FFF` (4KB window)

Planned Features: - 32.768 kHz clock input (typical RTC crystal frequency) - Seconds, minutes, hours, day, month, year tracking - Alarm functionality - Battery backup support (power domain considerations) - 24-hour or 12-hour (AM/PM) mode - Leap year handling - APB register interface

Applications: - System time-of-day tracking - Wake-on-alarm functionality - Timestamp generation - Power-aware applications

5.3.5 3.5 SMBus Controller

Status: Implemented (see dv/tests and the block MAS; per-block defects tracked in known_issues/) **Priority:** Medium **Address:** `0xFEC0_4000 - 0xFEC0_4FFF` (4KB window)

Planned Features: - SMBus 2.0 compliance - Master and slave modes - Clock stretching support - Packet Error Checking (PEC) - Alert response address - Configurable clock speed - APB register interface

Applications: - System management bus communication - Sensor interfaces (temperature, voltage) - EEPROM access - Battery management - Fan control

5.3.6 3.6 UART - Universal Asynchronous Receiver/Transmitter

Status: Implemented (apb4_uart_16550 + MAS; see docs/uart_16550_mas – note the shipped block has 1/2 stop bits only and no automatic flow control, and issue #60 tracks its RTL defects) **Priority:** Medium **Address:** `0xFEC0_8000 - 0xFEC0_8FFF` (4KB window)

Planned Features: - 16550-compatible register interface - Configurable baud rate generation - 5/6/7/8 data bits - Parity: none, even, odd, mark, space - Stop bits: 1, 1.5, 2 - Hardware flow control (RTS/CTS) - FIFO buffers (16-byte TX/RX) - Interrupt generation

Applications: - Debug console - Serial communication - Modem interfaces - Legacy peripheral communication

5.3.7 3.7 SPI Controller

Status: Planned **Priority:** Low **Effort:** 5-6 weeks **Address:** TBD (not in primary ILB address map)

Planned Features: - Master mode (initially; slave mode future) - Configurable clock polarity and phase (CPOL/CPHA) - Multiple chip selects - Configurable word size (8/16/32 bits) - TX/RX FIFOs - DMA support (future) - APB register interface

Applications: - Flash memory access - ADC/DAC interfaces - Display controllers - SD card communication

5.3.8 3.8 I2C Controller

Status: Planned **Priority:** Low **Effort:** 5-7 weeks **Address:** TBD (not in primary ILB address map)

Planned Features: - I2C standard (100 kHz), fast (400 kHz), fast-plus (1 MHz) modes - Multi-master arbitration - 7-bit and 10-bit addressing - Clock stretching - General call support - APB register interface

Applications: - Sensor interfaces - EEPROM access - Multi-chip communication - System configuration

5.3.9 3.9 Watchdog Timer

Status: Planned **Priority:** Low **Effort:** 2-3 weeks **Address:** TBD (not in primary ILB address map)

Planned Features: - Configurable timeout period - Countdown counter with reload - Reset generation on timeout - Lock mechanism to prevent accidental disable - Interrupt before reset (optional warning) - APB register interface

Applications: - System fault recovery - Software hang detection - Periodic system reset - Safety-critical applications

5.3.10 3.10 Power Management / ACPI Controller

Status: Implemented (see dv/tests and the block MAS; per-block defects tracked in known_issues/) **Priority:** Medium **Address:** `0xFEC0_5000 - 0xFEC0_5FFF` (4KB window)

Planned Features: - Clock gating control per block - Power domain sequencing - Reset generation and distribution - Wake event handling - Sleep/idle mode control - ACPI-compatible registers - APB register interface

Applications: - Low-power system design - Battery-powered devices - Dynamic power management - Thermal management - OS power management interface

5.3.11 3.11 IOAPIC - I/O Advanced Programmable Interrupt Controller

Status: Implemented (see dv/tests and the block MAS; per-block defects tracked in known_issues/) **Priority:** Medium **Address:** `0xFEC0_6000 - 0xFEC0_6FFF` (4KB window)

Planned Features: - I/O APIC CSR model (register-based interface) - Multiple interrupt inputs (24+) - Programmable interrupt routing - Edge and level triggered modes - Priority-based arbitration - Interrupt masking per input - APB register interface for configuration

Applications: - Advanced interrupt routing - Multi-processor interrupt distribution - Flexible interrupt mapping - Legacy IRQ redirection - PC-compatible systems

5.3.12 3.12 Interconnect ID / Version Registers

Status: Planned – NOT present in the shipped design **Priority:** Low **Effort:** 1-2 weeks **Address:** unallocated. The shipped decoder is the generated `apbx_xbar_1to10`, whose ten windows

(`0x0000-0x9FFF`) are fully assigned, so adding this block needs a window assignment AND a crossbar regeneration – it cannot simply occupy `0xF000` as earlier drafts of this document assumed.

Planned Features: - Vendor ID register - Device ID register - Revision ID register - Block presence/capability bits - Configuration status registers - Debug/diagnostic registers - APB register interface

Applications: - Software block discovery - Version checking - Feature detection - Debug and diagnostics - Platform identification

---

5.4 4. Integration and Wrapper Goals

5.4.1 4.1 Individual Block Integration

Each block is designed to be used standalone:

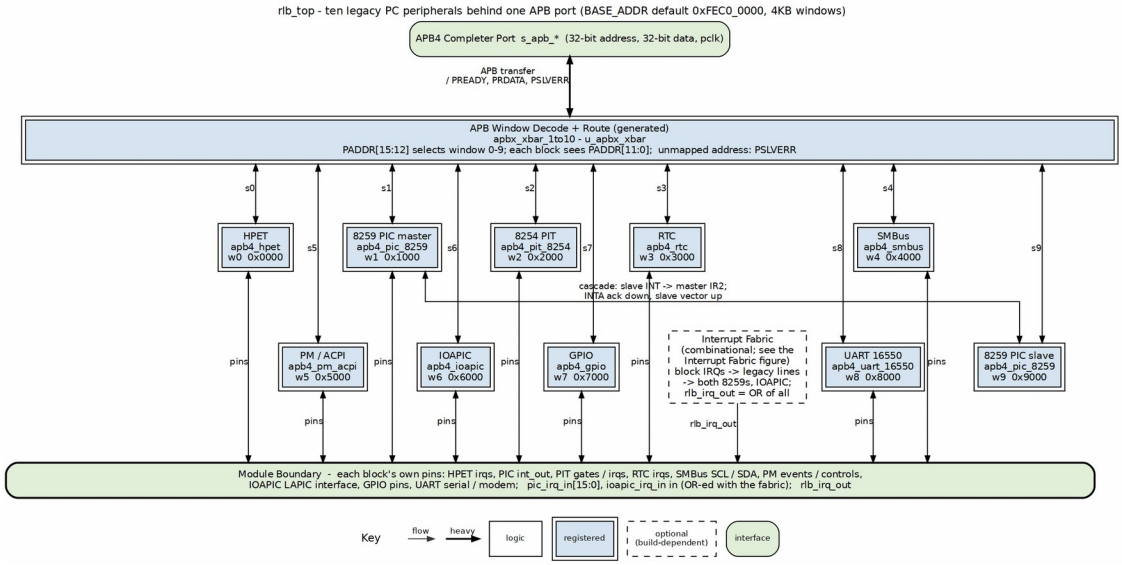
Example - HPET Integration:

```
apb4_hpet #(
    .NUM_TIMERS(3),
    .VENDOR_ID(16'h4747),
    .REVISION_ID(16'h0001),
    .CDC_ENABLE(0)
) u_hpet (
    .pclk           (apb_clk),
    .presetn        (apb_rst_n),
    // APB interface -- the ports are s_apb_P*, not bare p*
    .s_apb_PSEL     (psel_hpet),
    .s_apb_PENABLE  (penable),
    .s_apb_PREADY   (pready_hpet),
    .s_apb_PADDR    (paddr[11:0]),
    .s_apb_PWRITE   (pwrite),
    .s_apb_PWDATA   (pwdata),
    .s_apb_PSTRB    (pstrb),
    .s_apb_PPROT    (pprot),
    .s_apb_PRDATA   (prdata_hpet),
    .s_apb_PSLVERR  (pslverr_hpet),
    // HPET-specific
    .hpet_clk       (timer_clk),
    .hpet_resetn    (timer_rst_n),
    .timer_irq    (timer_irq[2:0])
);
```

5.4.2 4.2 RLB Wrapper Architecture

Goal: Create top-level wrapper combining multiple legacy blocks into unified retro-compatible subsystem.

System Architecture:



Address Map:

Base address: `0xFEC0_0000` – the `BASE_ADDR` parameter default in `rlb_top.sv`, and the conventional location for this class of block. It is a PARAMETER: an integrator may relocate the whole window set. Window size: 4KB per block (clean power-of-2 decode)

Address Range	Block	Size	Function
`0xFEC0_0000 - 0xFEC0_0FFF`	HPET	4KB	High Precision Event Timer
`0xFEC0_1000 - 0xFEC0_1FFF`	8259 PIC (master)	4KB	Programmable Interrupt Controller
`0xFEC0_2000 - 0xFEC0_2FFF`	8254 PIT	4KB	Programmable Interval Timer
`0xFEC0_3000 - 0xFEC0_3FFF`	RTC	4KB	Real-Time Clock
`0xFEC0_4000 - 0xFEC0_4FFF`	SMBus	4KB	SMBus Host Controller
`0xFEC0_5000 - 0xFEC0_5FFF`	PM/ACPI	4KB	Power Management / ACPI Registers
`0xFEC0_6000 - 0xFEC0_6FFF`	IOAPIC	4KB	I/O Advanced PIC (CSR model)
`0xFEC0_7000 - 0xFEC0_7FFF`	GPIO	4KB	General Purpose

Address Range	Block	Size	Function
			I/O
0xFEC0_8000 - 0xFEC0_8FFF	UART 16550	4KB	Serial port
0xFEC0_9000 - 0xFEC0_9FFF	8259 PIC (slave)	4KB	Cascade slave of the PC/AT pair
All other addresses	decode-miss agent	-	Accepted and answered locally with PSLVERR

The decoder is the GENERATED `apbx_xbar_1to10` (one master, ten slaves), so the window count and the base both come from the generator invocation recorded in `rlb_top.sv`. There is no window above `0x9FFF`.

Decoder Implementation:

```
// Address decode logic (simplified)
localparam BASE_ADDR = 32'hFEC0_0000;
localparam BLOCK_SIZE = 12;  // 4KB = 2^12

logic [3:0] block_sel;
assign block_sel = paddr[15:12];  // Extract window number

always_comb begin
    psel_hpet      = (block_sel == 4'h0) & psel;  // 0xFEC0_0xxx
    psel_pic8259   = (block_sel == 4'h1) & psel;  // 0xFEC0_1xxx
    psel_pit8254   = (block_sel == 4'h2) & psel;  // 0xFEC0_2xxx
    psel_rtc       = (block_sel == 4'h3) & psel;  // 0xFEC0_3xxx
    psel_smbus     = (block_sel == 4'h4) & psel;  // 0xFEC0_4xxx
    psel_pm        = (block_sel == 4'h5) & psel;  // 0xFEC0_5xxx
    psel_ioapic    = (block_sel == 4'h6) & psel;  // 0xFEC0_6xxx
    psel_gpio      = (block_sel == 4'h7) & psel;  // 0xFEC0_7xxx
    psel_uart      = (block_sel == 4'h8) & psel;  // 0xFEC0_8xxx
    psel_pic_slave = (block_sel == 4'h9) & psel;  // 0xFEC0_9xxx
    psel_error     = !(|{psel_hpet, psel_pic8259, psel_pit8254,
                         psel_rtc, psel_smbus, psel_pm, psel_ioapic,
                         psel_gpio, psel_uart, psel_pic_slave}) &
psel;
end
```

Interface: - **Single APB slave port** at base address `0xFEC0_0000` (the `BASE_ADDR` parameter) - **Aggregated interrupt output** combining all block IRQs - **Per-block clock/reset control** for power management - **External I/O signals** (GPIO, UART, I2C/SMBus, etc.) - **Error slave** returns SLVERR for unmapped addresses

Benefits: - Simplified system integration (single APB slave) - Consistent 4KB window addressing - Clean power-of-2 address decode - Ten 4KB windows, fully assigned; expansion needs a crossbar regeneration - Single verification target - Drop-in retro-compatible peripheral subsystem

---

5.5 5. Design Standards

5.5.1 5.1 Reset Handling

MANDATORY: All blocks must use standardized reset macros from `rtl/amba/includes/reset_defs.svh`

Pattern:

```
`include "reset_defs.svh"

`ALWAYS_FF_RST(clk, rst_n,
    if (`RST_ASSERTED(rst_n)) begin
        r_state <= IDLE;
        r_counter <= '0;
    end else begin
        r_state <= w_next_state;
        r_counter <= r_counter + 1'b1;
    end
)
```

Why: - FPGA-friendly reset inference - Consistent synthesis behavior - Single-point reset polarity control - Better timing closure

5.5.2 5.2 Register Generation

Preferred: Use PeakRDL for register map generation

Process: 1. Define registers in SystemRDL (`.rdl` file) 2. Generate RTL using PeakRDL regblock 3. Create wrapper module connecting registers to core logic 4. Use edge detection for write strobes (not level)

Benefits: - Consistent register interface - Auto-generated documentation - Reduced manual RTL errors - Easy register map changes

5.5.3 5.3 Testbench Architecture

MANDATORY: Follow project testbench organization pattern

Structure:

```
dv/
├── tbclasses/{block}/          # Block-specific TB classes
│   ├── {block}_tb.py          # Main testbench
│   ├── {block}_tests_basic.py # Gate test suite
│   ├── {block}_tests_medium.py # Func test suite
│   └── {block}_tests_full.py  # Full test suite
└── tests/{block}/             # Test runners
    ├── test_apb4_{block}.py   # Pytest wrapper
    └── conftest.py            # Pytest configuration
```

Import Pattern:

```
# Always import from PROJECT AREA
from projects.components.retro_legacy_blocks.dv.tbclasses.{block}.
{block}_tb import {Block}TB
```

Test Levels: - **Gate:** Core functionality (register access, basic operation) - **Func:** Extended features (modes, configurations, edge cases) - **Full:** Stress testing, CDC variants, corner cases

Target: 100% pass rate at all levels

5.5.4 5.4 FPGA Synthesis Attributes

MANDATORY: Add FPGA synthesis hints for memory arrays

```
`ifdef XILINX
    (* ram_style = "auto" *)
`elsif SYNTH_PRAGMA
    /* synthesis ramstyle = "AUTO" */
`endif
logic [DATA_WIDTH-1:0] mem [DEPTH];
```

5.5.5 5.5 Documentation Requirements

Each block must have: - RTL comments (inline) - Register map specification - Block-level specification in `docs/{block}_mas/` - Integration guide - Test plan and results

---

5.6 6. Quality Metrics

5.6.1 6.1 Production Readiness Criteria

A block is considered “Production Ready” when:

- All gate tests pass 100%
- All func tests pass 100%
- All full tests pass ≥95%
- Complete register map specification
- RTL lint clean (Verilator)
- Reset macros used throughout
- FPGA synthesis attributes applied
- Integration guide written
- Known issues documented

5.6.2 6.2 Current Status

Block	Priority	Status	Test Pass Rate	Documentation	Production Ready
HPET	High	Implemented	see dv/tests	MAS (ch1/2/5)	RTL defects open (#46)
8259 PIC	High	Implemented	see dv/tests	MAS (ch1/5)	ISR/EOI half inert (#50)
8254 PIT	High	Implemented	6/6 both CDC configs	MAS (full)	Mode 0 only (#52)
GPIO	Medium	Implemented	see dv/tests	MAS (full)	Functional; quirks tracked (#44)
RTC	Medium	Implemented	see dv/tests	MAS (ch1/5)	Binary 24h only; time-set broken in production mode (#56)
SMBus	Medium	Implemented	see dv/tests	MAS (ch1/5)	Transfer engine non-functional (#58)

Block	Priority	Status	Test Pass Rate	Documentation	Production Ready
PM/ ACPI	Medium	Implemented	see dv/tests	MAS (ch1/5)	Status/W1C/wake defects (#54)
IOAPIC	Medium	Implemented	see dv/tests	MAS (ch1/2/5)	Double delivery + EOI hazards (#48)
UART	Medium	Implemented	see dv/tests	MAS (full)	FE/BI dead, IER unwired (#60)
SPI	Low	Planned	N/A	N/A	No
I2C	Low	Planned	N/A	N/A	No
Watchdog	Low	Planned	N/A	N/A	No
Interconnect	Low	Planned	N/A	N/A	No

5.7 7. Development Roadmap

5.7.1 7.1 Phase 1: Foundation (Complete)

- HPET implementation
- Directory structure for multiple blocks
- Testbench architecture established
- Documentation templates
- Build and test infrastructure

5.7.2 7.2 Phase 2: Core Peripherals (Next 6-9 Months)

Q1 2026 (High Priority): - 8259 PIC (6-8 weeks) - Interrupt controller - 8254 PIT (4-5 weeks) - Interval timer - RTC (3-4 weeks) - Real-time clock

Q2 2026 (Medium Priority): - GPIO Controller (4-6 weeks) - SMBus Controller (6-8 weeks) - PM/ACPI Controller (8-10 weeks)

Q3 2026: - UART (4-5 weeks) - IOAPIC (6-8 weeks)

5.7.3 7.3 Phase 3: Advanced Peripherals (9-15 Months)

Q4 2026: - SPI Controller (5-6 weeks) - I2C Controller (5-7 weeks) - Watchdog Timer (2-3 weeks)

Q1 2027: - Interconnect ID/Version Registers (1-2 weeks) - ILB Wrapper integration starts

5.7.4 7.4 Phase 4: System Integration (15+ Months)

Q2-Q4 2027: - Complete ILB wrapper with all blocks - System-level integration examples - Performance characterization - FPGA reference designs - Application notes - Software driver examples

---

5.8 8. References

5.8.1 8.1 External Standards

Peripheral Specifications: - ACPI HPET Specification 1.0a - SMBus Specification Version 2.0 - 16550 UART Datasheet - I2C Specification (NXP) - SPI Protocol Specification

Bus Protocols: - AMBA APB Protocol Specification (ARM) - AMBA 3 APB Protocol v1.0

5.8.2 8.2 Internal Documentation

- `/CLAUDE.md` - Repository AI guide
- `/PRD.md` - Master repository requirements
- `projects/components/legacy-ip/retro_legacy_blocks/CLAUDE.md` - Component AI guide
- `projects/components/legacy-ip/retro_legacy_blocks/README.md` - Component overview
- `vault/tracking/projects/components/legacy-ip/retro_legacy_blocks/` - task tracking (cross-block); `vault/tracking/projects/components/legacy-ip/retro_legacy_blocks/hpet/` for HPET

5.8.3 8.3 Block-Specific Documentation

HPET: - `docs/hpet_mas/hpet_mas_index.md` - HPET specification - `docs/IMPLEMENTATION_STATUS.md` - HPET test results - `known_issues/` - HPET issue tracking

---

5.9 9. Success Criteria

5.9.1 9.1 Individual Block Success

Each block must: - Pass all gate/func tests at 100% - Pass full tests at ≥95% - Have complete register map specification - Include integration guide with examples - Be lint-clean (Verilator) - Use reset macros throughout - Include FPGA synthesis attributes

5.9.2 9.2 Collection Success

The retro_legacy_blocks component is successful when: - At least 6 blocks production-ready (HPET + 5 high/medium priority blocks) - All blocks follow consistent architecture (reset macros, PeakRDL, APB interface) - RLB wrapper integrates all blocks seamlessly with clean 4KB addressing - System-level integration example provided - Complete documentation for all blocks - FPGA reference design available - Address map covers all essential retro-compatible peripherals

5.9.3 9.3 Long-Term Vision

Ultimate goal: - Production-quality retro-compatible peripheral subsystem - Complete peripheral coverage for legacy platform requirements - Used in production FPGA designs - Educational resource for classic peripheral design - Foundation for mixed-vintage SoC designs

---

Version: 1.0 **Last Review:** 2025-10-29 **Next Review:** After each new block completion **Maintained By:** RTL Design Sherpa Project